An Open Data Service for Supporting Research in Machine Learning on Tokamak Data

Samuel Jackson, Saiful Khan, Nathan Cummings, James Hodson, Shaun de Witt, Stanislas Pamela, Rob Akers, Jeyan Thiyagalingam, *Senior Member, IEEE*, and MAST Team

***Abstract*—The increasing complexity and volume of plasma fusion experimental data, coupled with the growing adoption of machine learning in fusion research, necessitate advanced and efficient data management solutions. We propose an open data service for fusion experiments operated by the UKAEA, designed to address the evolving needs of machine-learning-driven fusion research. Our system provides a framework to organize MAST, MAST upgrade (MAST-U), and Joint European Torus (JET) experimental data in accordance with findability, accessibility, interoperability, and reuse (FAIR) principles, using distributed object storage for scalability and a relational database for efficient metadata indexing. In addition, it offers simplified abstractions through an application programming interface (API), facilitating seamless data access and integration with data analysis and machine learning workflows. Performance evaluation of metrics such as data load time and throughput, across varying numbers of parallel workers, demonstrates the data pipeline's optimization for efficient machine learning application development. Our solution significantly enhances support for data-driven research and machine learning applications in fusion by laying the groundwork for open, FAIR-compliant fusion data, which enables cross-machine analysis, prompts international collaboration, and potentially accelerates advancements in fusion energy research.**

***Index Terms*—Fusion data, scientific data management, web service application programming interface (API).**

I. Introduction

THE increasing complexity and volume of data generated by fusion experiments necessitate robust, scalable, and efficient data management systems. With the development of larger tokamak facilities, such as international thermonuclear experimental reactor (ITER) and DEMOnstration power plant (DEMO) [1], [2], which will produce longer pulses in pursuit of energy production, it is crucial for the fusion research community to work toward modern, scalable data solutions.

In the meantime, the field of fusion energy and plasma science is transitioning into the fourth paradigm of data-intensive science [3], where data-driven models and machine learning play an increasingly prominent role [4], [5]. Data-driven models offer potential for workflow automation, control system optimization, data compression, data visualization, uncertainty quantification, and surrogate modeling. Anirudh et al. [4] highlight the promise of these techniques across numerous areas of fusion research. Moreover, the recent success of foundation models in natural language processing and computer vision [6], [7] has sparked interest in developing domain-specific models for physical sciences. However, data-driven models rely on large volumes of analysis-ready data and public availability of such data is essential to foster transparency, collaboration, and innovation.

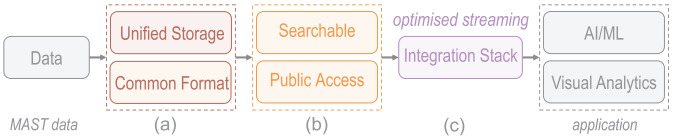

Fig. 1. Our system transforms archived tokamak data into a format suitable for downstream data analytics and machine learning applications. (a) To prepare the data for analysis, we unify its storage location and standardize on a common format. (b) We enhance findability and accessibility through APIs for searching and filtering, while also ensuring public access data and metadata. (c) To support integration with analysis workflows and machine learning pipelines, our solution integrates seamlessly with commonly used data analysis tools.

MAST [8], [9], [10] was a spherical tokamak commissioned by Euratom/UKAEA and based at Culhum Centre for Fusion Energy. MAST operated from 1999 to 2013, and MAST upgrade (MAST-U) [11] continues its legacy. The Joint European Torus (JET) was operational between 1983 and 2023, generating over 40 years of data on fusion research. To fully leverage the opportunities for data-driven and machine learning approaches, it is essential to curate, transform, and, where possible, make these data publicly accessible using open-source, high-performance tools for analysis, such as those in the Pandata stack [12].

This article proposes an innovative data service and framework tailored for fusion data, that adheres to findability, accessibility, interoperability, and reuse (FAIR) principles [13]. Our goals are to: 1) transform historical tokamak data into analysis-ready data products; 2) provide public access and ensure findability of the historical data; and 3) enable integration with data analytics and machine learning pipelines via open-source tools. Fig. 1 illustrates how these objectives are achieved. To meet (a), the system consolidates data from MAST, MAST-U, and JET into a unified storage solution, ensuring all the data objects adhere to a standardized, interoperable format with linking to the integrated modeling and analysis suite of ITER (IMAS) data dictionary. For (b), metadata are integrated into a searchable database, and in

Received 1 November 2024; revised 1 May 2025 and 28 May 2025; accepted 18 June 2025. This work was supported by UKAEA. The review of this article was arranged by Senior Editor P. Stoltz. *(Corresponding author: Samuel Jackson.)*

Samuel Jackson, Nathan Cummings, James Hodson, Shaun de Witt, Stanislas Pamela, and Rob Akers are with U.K. Atomic Energy Authority (UKAEA), Culham Centre for Fusion Energy, Culham Science Centre, OX14 3EB Abingdon U.K. (e-mail: samuel.jackson@ukaea.uk; nathan.cummings@ukaea.uk; james.hodson@ukaea.uk; shaun.de-witt@ukaea.uk; Stanislas.Pamela@ukaea.uk; rob.akers@ukaea.uk).

Saiful Khan and Jeyan Thiyagalingam are with the Scientific Computing Department, Science and Technology Facilities Council, Rutherford Appleton Laboratory, Harwell Campus, OX11 0QX Didcot, U.K. (e-mail: saiful.khan@stfc.ac.uk; t.jeyan@stfc.ac.uk).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TPS.2025.3583419.

Digital Object Identifier 10.1109/TPS.2025.3583419

the case of MAST, the curated dataset is made publicly available, ensuring worldwide accessibility without barriers. For MAST-U and JET data, which cannot yet be made public due to data embargo and policy constraints, we show how the developed solution also seamlessly handles MAST-U and JET data. Finally, (c) is achieved through the provision of thin application programming interfaces (APIs) and access layers that facilitate integration with common data analysis stacks, fostering collaboration with the broader open-source software community.

Data management for streamlining applications such as machine learning or visual analytics presents unique challenges due to the domain-specific characteristics of data involved. This article introduces a robust design and architectural framework specifically created for our MAST, MAST-U, and JET data. However, we believe that the underlying data management and service stack [Fig. 5(b)] can be adapted for application in other fields, thus broadening its utility for a wider range of scientific data management. To promote wider adoption and collaborative improvement, we have made our software open-source [14]. Our goal is to empower researchers and practitioners within the scientific community to use and improve our framework, while also aiming to foster innovation in data management practices beyond the fusion domain.

II. Related Work

A. Data Management

Within the fusion community, TokSearch [15], [16] is a tool for executing parallel queries over an archive of fusion experimental data. TokSearch is a complementary technology to the solution presented in this work. Our work provides the necessary platform from which we can integrate UKAEA data with domain-specific tools such as TokSearch.

Recent work [17] at the EAST tokamak proposes a data service similar to ours. The service consists of an NoSQL metadata database and store fusion data in hierarchical data format (HDF) files. It also includes a parallel data bucketing and buffering system for dataset creation. Our work is differentiated by: 1) we incorporate a FAIRification pipeline to help structure and standardize data toward IMAS interoperability; 2) we make the historical data (MAST) open and publicly available; and 3) we integrate with the Pandata [12] stack to leverage open-source tools, rather than develop and maintain domain specific solutions.

Recently, the large helical device (LHD) has published 25 years worth of experimental data [18] through the Amazon sustainability data initiative. However, these data are only provided in a mix of esoteric data formats that are not easily searchable through the metadata.

B. Metadata Standards

Plasma-MDS [19] is a set of metadata conventions for the plasma physics community. It is designed with FAIR data policies in mind. Plasma-MDS extends generic metadata formats such as Dublin Core [20], DCAT [21], and DataCite [22] with domain-specific descriptors. Specifically, they provide four additional entries describing the plasma source, medium, target, and diagnostics attached.

The IMAS data dictionary [23], [24] defines the data schema used by ITER. The IMAS interface data structures (IDSs) provide a hierarchical schema for a wide plethora of diagnostic signals. IMAS is the most relevant metadata standard to our work. In Section V, we describe how we integrate our data curated products with the IMAS data dictionary.

C. FAIR Fusion Data

The FAIR principles [13], [25] are a set of data management and policy guidelines introducing a number of benefits. First, open data promote transparency and trust in research findings. The FAIR principles enable advanced, multidisciplinary analyses and innovative solutions by making data analyzable across different platforms and software tools. This maximizes the return on investment for data collected through public funding. Interoperability fosters innovation and collaboration across various different scientific domains and minimizes "reinventing the wheel" in software development. Finally, FAIR data ensure that metadata and expert knowledge are captured as part of a digital asset, ensuring that data serve as a valuable asset for the scientific community and society at large.

Prior work by Strand et al. [26] established the need for FAIR principles to be applied to fusion data. Specifically, they noted the following recommendations.

1) *Remote Access:* They emphasized the need for remote access between academia/industry and experimental facilities to enable collaboration and engagement with the data.
2) *Metadata Mappings:* The need to develop common schemas and mapping tools, in particular toward IMAS [23], [24].
3) *Search and Discovery:* User-oriented tools for data searching, mining, and visualization are essential.
4) *Licensing:* Using a Creative Commons license (CC-BY-NC-SA) to make validated fusion data open to the public and the wider research community, with an embargo period for initial exploitation by the data producers.

While addressing all the points surveyed in [26] is well beyond the scope of a single work, our open data service builds toward these goals.

III. MAST, MAST-U, and JET Data: Challenges and Requirements

A. Introduction to MAST, MAST-U, and JET Data

Tokamaks produce thousands of shots throughout their lifetime. A shot is a single operational run of the tokamak. For example, during its operation MAST produced a total of 30 471 shots, JET took 105 929, while MAST-U has already taken over 10 000 shots during its six years of operation. Shots can be logically grouped into campaigns, sequences of multiple experimental sessions with specific science objectives. For example, there were a total of nine campaigns (M1–M9) for MAST, while MAST-U is already on its fourth campaign. Each shot may produce gigabytes of diagnostic data. The exact size and number of signals vary with the duration of the shot and the number/type of diagnostic devices attached. A single shot

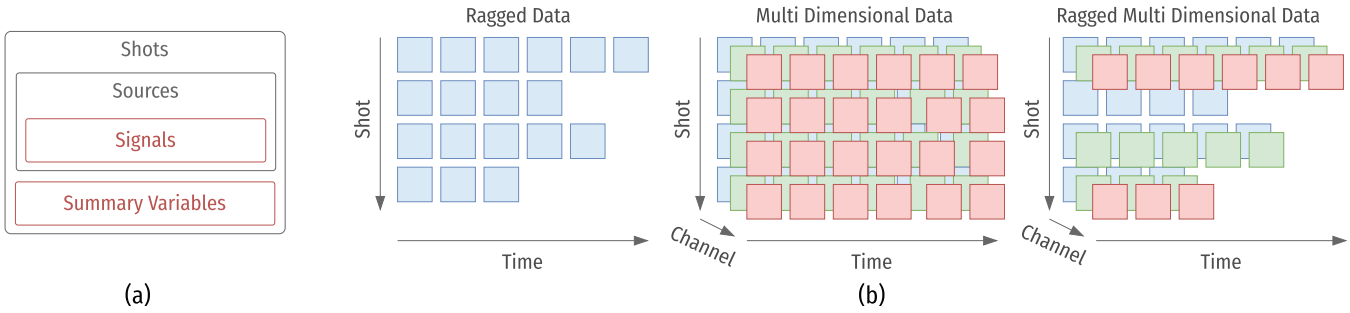

Fig. 2. (a) Schema of a shot (left). Each shot contains multiple data sources. A signal is a data item which can have any number of dimensions. Each dimension may be associated with a coordinate (e.g., time, major radius). In addition, the summary variables provide a statistical summary of the shot properties. The MAST experiment captured images from diagnostics in various dimensionalities and may include multidimensional ragged or irregular data, as shown in (b). *Ragged Data* are diagnostics on MAST that often record 1-D time-series signals where the time dimension varies between shots. *Multidimensional Data* are diagnostics that have additional dimensions other than time, such as the number of channels or height and width dimensions of a video. *Multidimensional Ragged Data* are diagnostics whose additional dimensions, such as channels, height, and width, vary between shots.

from MAST, for example, contains approximately ~7 GB of data when uncompressed.

Conceptually, tokamak data are organized hierarchically (Fig. 2). A dataset comprises many shots. A single shot may be associated with high-level summary statistics and parameters characterizing the nature of a shot. Each shot contains many diagnostic data sources (e.g., Thomson diagnostic, soft x-ray camera, and equilibrium), each of which consist of many signals (e.g., plasma current, electron temperature, and soft x-ray brightness). Signals may be 1-D time-varying quantities (e.g., plasma current), 2-D quantities (e.g., Thomson profiles with time versus major radius), or 3-D quantities (e.g., equilibrium magnetic flux map, camera video data).

Access to archived data is provided via domain-specific access layers, such as UDA [27] for MAST/MAST-U and SAL for JET. Using an access layer, users can extract signals and the accompanying metadata (such as name, units, and description). Each signal is represented by multiple arrays such "value" and "time." Additional important metadata may only be available elsewhere. For example, for MAST/MAST-U, shot-level metadata is only available in an internal Drupal website. To capture these data, we take dump of the database and extract the relevant metadata as part of the ingestion process.

B. Data Challenges

Tokamak data are challenging to work with. Data are stored in a mixture of file formats, including bespoke, legacy, binary formats. Much of the data were not written using a standardized ontology or schema, and without standardized or consistent names for variables, dimensions, or units.

Current domain-specific access layer (UDA/SAL), do not satisfy many of the goals outlined in Section I. Specifically, they do not make the data publicly accessible, do not support user-side caching, do not support complex lazy loading operations, may not support parallel data access, and do not integrate with open-source data analysis tools. Diagnostic data from tokamaks can be considered ragged multidimensional data [Fig. 2(b)]. Time-series data are often ragged: each observation has a different length in the time dimension. Multidimensional data refer to an array of data with more than 1-D, represented as a multidimensional matrix. Tokamak data exhibit both these features simultaneously: observations between shots can vary both in time and other dimensions (such as number of channels). This makes the data difficult to store in a performant and self-describing manner. Data may also vary significantly in size between different diagnostics. Fig. 3 shows the approximate distribution of diagnostic signal sizes within a representative shot from MAST (#30420). This makes it challenging to find a solution which works for all the data scales. The above issues combine to make the data challenging for downstream analysis and running machine learning workflows. Without standardized formats and a common ontology, the data are difficult to access and load across machines. The domain-specific and closed nature of the access layer codes leads to frequent issues accessing archive data. The absence of lazy loading, advanced search, and user-side caching often leads to data over-fetching. The absence of public access stifles collaboration within the fusion community and industrial partners. Furthermore, it makes reproducibility extremely challenging.

C. Data Service System Requirements

Considering these challenges, our proposed data service must effectively manage tens of thousands of experiments, accumulating to terabytes of data. It is crucial that the data can be quickly and reliably searched, filtered, and accessed for machine learning applications, visual analytics, and other data analysis tasks. Our developed, open-source system [28], comprehensively addresses the mentioned challenges and encompasses detailed software engineering requirements. We outline the high-level requirements here.

1) *R1:* Data and metadata must be publicly open and accessible over the web, with as few barriers to access as possible.
2) *R2:* Data access must support lazy loading and slicing of any dimension.
3) *R3:* Data access must support user-side caching, to prevent overfetching and reduce latency.
4) *R4:* The data format must provide multilanguage support for common scientific and analysis languages (e.g., Python, R, Julia, C, and Fortran) but prioritize Python.

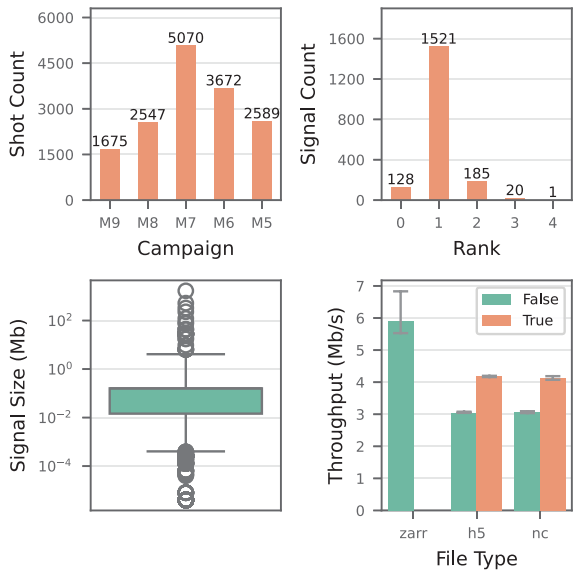

Fig. 3. Number of shots fired in each experimental campaign (top left). The M7 campaign, which took place between 2008 and 2011, was the longest campaign, recording over 5000 shots. Counts of the different numbers of dimensions of each signal in the experimental shot no. 30420 (top right). The majority of signals are 1-D, but data can have up to 4-D. Size of each signal example shot #30420 (bottom left). Signal sizes within a shot range from 10 bytes to 1 GB. Throughput of loading camera data of the central column of MAST instrument (bottom right). We tried HDF and NetCDF formats both with and without kerchunk. Error bars shows the standard deviation in sample throughput.

5) *R5:* The data format must be self-descriptive and contain the relevant metadata.
6) *R6:* The solution must integrate with tools that support larger-than-memory and parallel computation.
7) *R7:* The metadata must be quickly and easily searchable without downloading all the data.
8) *R8:* The proposed system must link with the existing data analysis and machine learning tool ecosystems.
9) *R9:* The data must be stored in a format that supports the complex, ragged, multidimensional, and multiscale nature of tokamak data.
10) *R10:* The solution should use open-source standards and protocols as much as possible.

Examining these requirements, it is clear that they include the definitions of FAIR data (Section II-C). R7 satisfies findability, R1 satisfies accessibility, R4 and R8 satisfy interoperability, and R5 satisfies reusability. They also encompass many of the requirements highlighted by Strand et al. [26].

IV. Design Options

A. Array Data Storage

The first challenge is to choose an appropriate array data storage technology for the data (R4, R5, and R9), while also allowing public accessibility to satisfy R1. We consider two broad categories for storing complex multidimensional data: array database management systems (array DBMSs) and hierarchical file formats in object storage.

Array DBMSs are an extension of traditional tabular DBMSs for storing and querying multidimensional array data. Well-known array DBMS technologies include RasDaMan [29], SciDB [30], [31], and TileDB [32]. Array DBMSs have found some adoption within the Earth science community, for example, [33]. Array DBMSs function like a traditional database, usually supporting an SQL-like syntax for querying data. They can provide complex tiling of data arrays in irregular and partially aligned grids (R9). They may also support distributed and parallel processing of user queries.

Hierarchical file formats are binary file formats that support complex multidimensional data (R9), as well as lazy loading (R2). Commonly used hierarchical formats include HDF [34], NetCDF [35], and Zarr [36]. They are self-describing and can store metadata in the file alongside the data (R5). In addition, they can support both the parallel reads and writes. To enable public access (R1), they may be combined with an object storage technology, such as Amazon's Simple Storage Service (S3).

S3 is a widely used protocol for accessing data in object storage. S3 stores data in buckets associated with a unique name and region. S3 supports access control layer (ACL) policies for fine-grained authentication and data versioning. Object storage has become increasingly common in other science domains, especially with the rise of initiatives such as the Amazon sustainability data initiative [18], [37].

Both the array DBMSs and hierarchical file formats are able to satisfy requirements R4, R5, and R9. However, we identify several reasons that make array DBMS less desirable compared with hierarchical file formats. First, array DBMSs use a central database for querying and selecting data. This puts an additional burden on the service provider to ensure sufficient hardware capability to handle user queries. In contrast, with hierarchical file formats querying is performed on the user side, offering integration opportunities with high-performance computing (HPC) systems.

Array DBMSs also introduce a layer of abstraction between the user and the data. The user must learn a domain-specific language dependent on the choice of DBMS used. This increases the learning curve and introduces an additional point of failure. In contrast, hierarchical file formats are language- and access-layer-independent. Furthermore, the fusion and tokamak science community is already somewhat familiar with HDF-like hierarchical file formats.

In addition, most array DBMSs provide little integration with modern data analysis solutions, in conflict with requirement R8. Only TileDB [38] offers integration with the xarray library [39], but the support is experimental and lacks all the necessary features we required. Furthermore, array DBMS can be limited in their multilanguage support (R4).

We chose to use a hierarchical file format, combined with an object storage for storing our data service. We compared reading camera data from three hierarchical file formats from object storage in Fig. 3. For the NetCDF and HDF file formats, we additionally explored using the Kerchunk [40] library to provide cloud-optimized access to data. Considering performance, we chose Zarr as the basis for our data products. We do not require the complexity of HDF, and its lack of support by tools such as xarray is a major negative. Between NetCDF and Zarr, Zarr offers faster read operations from a

cloud environment. Zarr also supports multiple backends and languages (R4). Furthermore, the installation and maintenance of parallel NetCDF libraries on HPC platforms is a frequent issue, while Zarr offers parallel operations out of the box. While Zarr is not yet as widely adopted, we feel the additional advantages offered over NetCDF outweight this disadvantage.

B. Metadata Storage

Searching and filtering metadata directly from files in object storage is challenging. Object storage is inefficient to iterate over and is designed for key-value lookup. Therefore, we propose to index the metadata in a separate database. This satisfies R7 to provide a service where users can query and filter to find relevant data without the need to directly download the file.

In database technology, an important dichotomy exists between SQL versus NoSQL databases. NoSQL systems store data in an unstructured form, while SQL system store data structured as tables. Examining the metadata for MAST fusion data in Section III, there are little unstructured data. Several relational database management systems, such as PostgreSQL, include some support for unstructured data (such as JSON types [41]) if required. Therefore, we chose to focus on SQL database systems.

Another key consideration is the choice of whether online analytical processing (OLAP) or online transaction processing (OTLP) is more important. OLAP-orientated databases are designed for large amounts of data processing, aggregation, and analysis. In contrast, OLTP-orientated databases are designed to handle a large number of transactions, with many insert, update, and delete operations happening simultaneously. Our proposed metadata database is required to support many users concurrently reading data for analysis. In addition, system administrators will also periodically update the contents with improvements. In the future, the system may be continually ingest data from MAST-U, where new data will be ingested daily. Therefore, the final system needs to primarily support OLAP operations, with OLTP operations as a secondary concern.

We considered several common relational database options such as PostgreSQL, MariaDB, and SQLite. We decided on PostgreSQL due to the combination of power, features, and reliability. Several relational databases, e.g., PostgreSQL and MariaDB, support column-oriented storage for OLAP processing while also supporting transactions and advanced data types such as arrays and JSON.

C. APIs, Access Layers, and Tool Integration

1) Metadata APIs: To meet requirements R1, R7, and R10, our system needs to provide methods for the user to interact with the metadata over a common and widely supported protocol. We considered two choices: REST and GraphQL.

REST APIs are a ubiquitous standard for web services. REST APIs leverage the HTTP protocol which is widely understood and supported. The statelessness provided by REST ensures there is a clear separation of concerns between services. Relying on HTTP as the protocol also makes REST flexible and able to return a variety of different formats, such as JSON, XML, or HTML.

GraphQL is a query language developed by Facebook (now Meta) [42]. GraphQL has several features that distinguish it from REST. First, GraphQL relies on a single endpoint, which contrasts with the proliferation of endpoints typically required by REST. In addition, GraphQL can combat overfetching because in a GraphQL query, the user must explicitly specify the fields they require. GraphQL query can fetch multiple data objects in one query, instead of multiple queries to different endpoints. Finally, GraphQL also enforces a strict, predefined schema for user interaction with the endpoint. The disadvantages of GraphQL are that it is not widely used, and many potential users may not have encountered it. In addition, GraphQL queries tend to be larger and more complex in comparison to a REST query.

Both the API schemes satisfy our requirements. In our system, we implemented endpoints for both the APIs using the same internal object relational mapping and minimal code duplication.

V. System Architecture and Design

A. Architecture

We follow a service-oriented [43], [44], [45] design for our system which are well-suited to the goals of our project (Section III-C). A service-orientated architecture prompts the use of loosely coupled services that are accessible over standard protocols. An overview of our architecture is shown in Fig. 4 and consists of the following components.

1) Object stores containing datasets of self-describing, hierarchical files with data recorded from the experimental history (R1, R2, R4, R5, and R9). For MAST data, the object storage is public. For MAST-U and JET, we use an internal object store in accordance with the limitations of current data policies.
2) A metadata database indexing the object storage with all the metadata from the datasets (R7). This single metadatabase includes data from all tokamaks ingested into the system, regardless of storage location, enabling federated search across machines.
3) Ingestion pipelines for transforming data sources to a standardized and FAIR representation (R4, R5, and R9).
4) API services for user access to the data and metadata using common protocols and tools (R1, R2, R3, R6, R7, R8, and R10).
5) An indexer component, responsible for gathering assorted metadata, harmonizing the names and ingesting them into the metadata database (R7).

B. Data Ingestion

To transform each shot from its source data format, we developed FAIR ingestion pipelines (Fig. 5) for each tokamak. Each diagnostic source has a pipeline applied to standardize data into the chosen file format. Within a pipeline, each signal is mapped to the relevant output group. Each pipeline is built with composable and reusable transformations. We apply four sets of mappings to standardize the data. These are as follows.

1) *Dimension Mappings:* These mappings rename the dimensions of each variable to use a common standard. We rename dimensions following IMAS conventions (e.g., for Thomson data, `r` is mapped

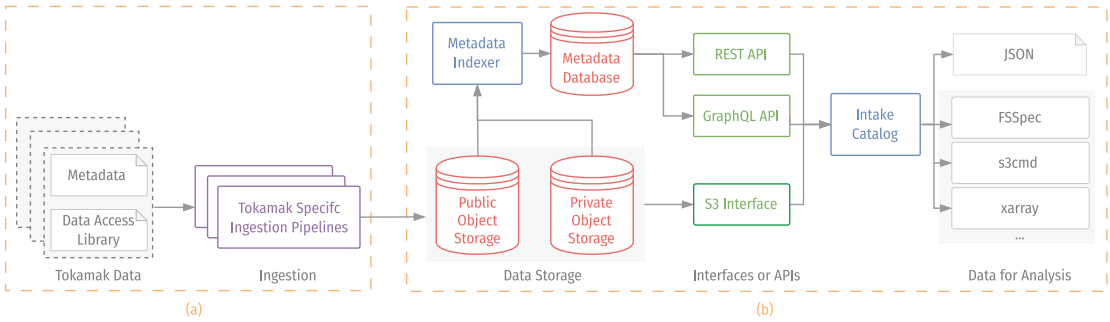

Fig. 4. Overview of our system architecture. (a) We transform and FAIR-ify raw data and metadata, sourced from different legacy systems, using an ingestion pipeline indexing module, and uploading to public storage services (see Fig. 5 for more details). (b) We support and provide standardized access protocols such as S3 and REST to access these services, adapted from [14]. Finally, an intake catalog provides a thin, gracefully degrading, access layer to simplify user access.

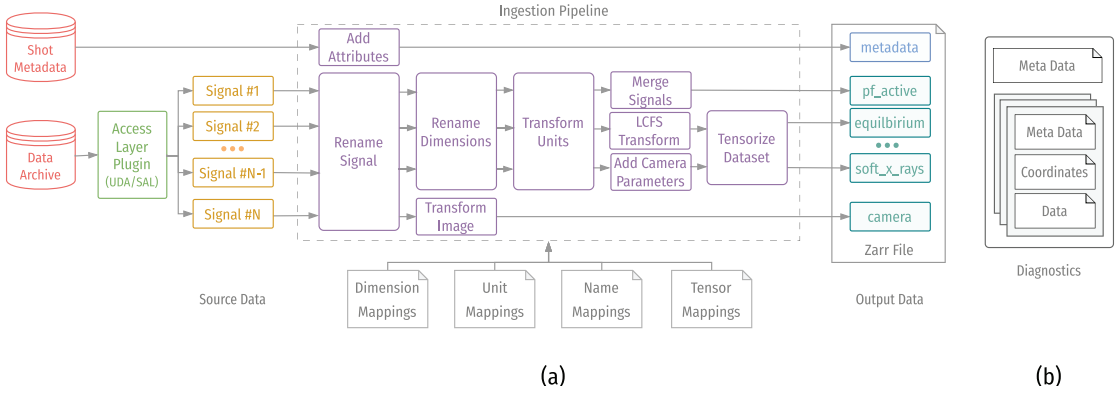

Fig. 5. (a) Detailed look at the data ingestion pipeline for MAST/MAST-U/JET data. Each tokamak has a custom pipeline. Data are sourced from the internal storage systems at UKAEA. We developed a unique ingestion pipeline for each source which uses composable data transformations. During the ingestion, we harmonize variable names, dimension names, and units. We also tensorize arrays of signals which can be sensibly grouped into higher dimensional arrays. (b) Data structure in each shot file. Each shot has multiple data sources and may have its own metadata. Each source contains metadata, coordinates, data, and optionally errors for various different signals.

to `major_radius`). Inconsistencies such as "Time," "timesec," and "time (s)" are all mapped to the name "time."

2) *Unit Mappings:* These mappings use the pint [46] library to standardize unit representations to follow IMAS specifications where possible. We also extend the pint unit registry where necessary to add specific parser definitions to capture edge cases.
3) *Name Mappings:* As closely as possible, we follow the naming conventions from the IMAS data dictionary source and signal names. We link signals to the IMAS IDS names in two ways: a) we add an `imas` metadata attribute to the signal with the full IMAS IDS path and b) we rename each signal to follow a short version of the IMAS IDS name. For example, `magnetics/ip/data(:)` is mapped to the source `magnetics` and with signal name `ip`).
4) *Tensor Mappings:* We combine multichannel variables into a single multidimensional matrix. For example, UDA flux loop flux data are available as separate channels: `AMB_FL/CC01`, `AMB_FL/CC02`, `AMB_FL/CC03` etc. each with shape $(1000,)$. These can be efficiently grouped as a single variable `flux_loops_flux`, with a new channel axis with shape $(3, 1000)$.

Additional data attached to signals, such as error arrays or data quality masks, are treated as separate signals and are appropriately mapped to their own entry within a Zarr group. Signals may be duplicated in multiple sources if this is specified within relevant IMAS IDS. For example, both the `magnetics` and `summary` groups contain `ip`. After applying our workflow for each source, we ingest the final file into object storage. The Zarr format can write out each diagnostic group independently and in parallel. After a file has been ingested, we parse the metadata for each shot and insert it to our metadata database using the metadata indexer.

C. Metadata Database Design

1) Metadata APIs and Data Access: We provide two APIs wrapping the metadata database: a REST API and a GraphQL API. Both the APIs reference the same underlying database tables, to give the user options for how to access their data. Detailed examples of the two APIs are shown in [14]. We reuse the same object relational mapping that is used for the REST API, meaning there is very little overhead to supporting both the APIs.

For access to the data, we host an intake catalog server [47] alongside our APIs and project documentation. We provide catalog entries both for loading metadata from the API and loading diagnostic data from Zarr files in object storage. We

enable user-side caching, meaning that once a user has loaded data from the bucket, it is cached locally for future access.

2) Database Tables: We use a PostgreSQL database for our implementation. Multiple tables are used to hierarchically index the data stored at three different levels: shots, sources, and signals. The relationship between these three concepts are described in Section III. Tables are shared across facilities and are tagged with a record identifying which facility the data item is associated with (e.g., MAST, MAST-U, and JET).

Shots: Each row in the shots table contains the information about a single shot, including the shot ID, timestamp, and information about the machine setup. In addition, it includes a number of summary variables such as confinement time, plasma temperature, and plasma beta which are useful for filtering.

Sources: The sources table contains metadata about groups of diagnostics signals for a shot. We define data sources by following IMAS IDS names. For example, signals from the soft X-ray cameras are grouped together in a source called `soft_x_rays`. Each row in the table indexes a single group within a shot file (Fig. 6). This is useful as a user may wish to only access a subset of the diagnostics from different shots (R2).

Signals: Signals are individual variables within each source. Each signal has metadata which can vary between shots, such as descriptions, shapes, and dimensions. Signals can be directly queried in the meta-database without the need to query the source a signal belongs to.

Shots, sources, and signals can be queried independently. The user does not need to query a source to find signals. Each database table is exposed with its own API endpoints, enabling searching and filtering at three levels of granularity.

D. Data Storage Design

1) File Format: We chose Zarr as our file format (R2, R4, R8, R9, and R10). The proposed file structure is shown in Fig. 5. Each file represents a single shot with groups for each diagnostic source. Sources contain data, coordinates, and metadata making each file self-describing (R5). A source may have heterogeneous time bases and coordinates, but shared coordinates are not duplicated between signals within a data product.

We link to the IMAS data dictionary schema via attributes attached to each variable (Section V-B). We deliberately avoid fully matching the IMAS data structure for two reasons: 1) strict adherence to the IMAS data structures cannot be directly represented in a hierarchical file formats without some form of mapping to the file format and 2) multiple instances of the same data item may cause namespace clashes (e.g., core and divertor Thomson scattering data in MAST-U).

After uploading to object storage, we index the metadata from each data product in the metadata database. Fig. 6 shows how the shot and source tables in our database link to the data through object storage urls. This enables users to find both the relevant file and the relevant entry within each file.

2) Object Storage: We store each shot file in an object store. Access to object storage is provided via the S3 protocol (R1). Our public object storage service is currently a Ceph

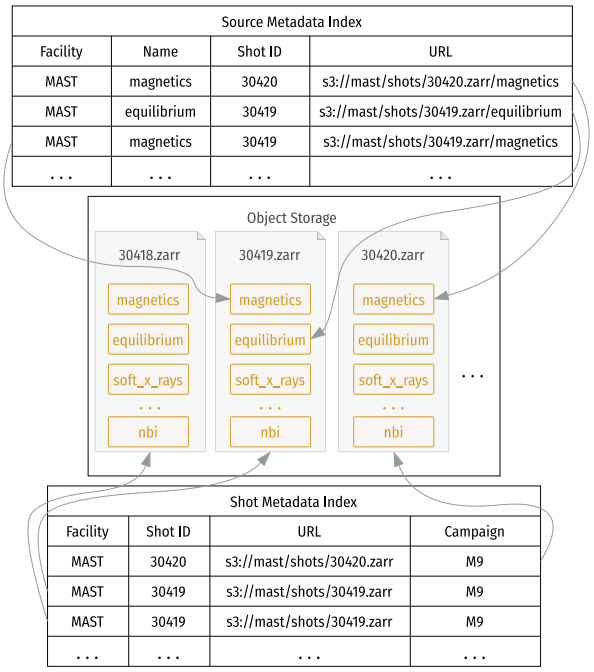

Fig. 6. Example of how our metadata database provides a loosely coupled, searchable index for data stored in the object store. Each row in the database contains a URL pointing to a specific piece of data. The database indexes the data hierarchically, the shot table points to specific files. The sources table points to specific groups with each file, which can be directly accessed.

cluster [48] hosted by the Science and Technology Facilities Council (STFC), while our private object storage is an internal Ceph instance at UKAEA. For public data, we allow anonymous public read access to make the data completely open, without any authentication. In future, we may support authenticated data access (e.g., data in an embargo period) by applying a more complex ACL policy.

VI. Performance Analysis

In a machine learning application, we require the ability to load multiple data samples in batches. Therefore, efficiently loading multiple data samples quickly is highly desirable. We consider performance from two perspectives: 1) by scaling the number of workers and 2) by lazy loading using the Dask [49] parallel computing library on a subset of data. Lazy loading defers data loading until it is explicitly required, reducing latency and optimizing memory usage, which can be advantageous when processing data over a network and can maximize the effectiveness of concurrence using tools such as asyncio. Conversely, eager loading fetches all the data, incurring a higher latency penalty when only a subset of data is required.

Fig. 7 shows our system's performance, measured in terms of latency and throughput of loading ten frames of tokamak pedestal data. The size of video depends on the shot duration, resulting in an average video size of around 100 MB. Scalability was assessed by varying the number of workers. To demonstrate the compatibility of our system with widely used machine learning libraries, we used the PyTorch [50] data loader and conducted the tests on UKAEA's internal data analysis cluster.

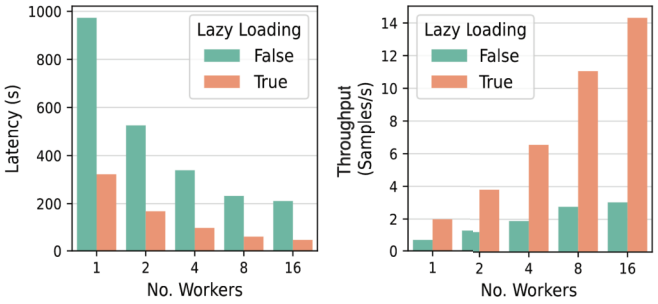

Fig. 7. Performance comparison of our system using lazy loading and eager loading of 640 camera images with varying numbers of parallel workers. The "latency" metric represents the total time taken to load all the images, and the "throughput" denotes the number of images loaded per second.

Our system demonstrates faster loading times [Fig. 7 (left)] and higher throughput [Fig. 7 (right)] with lazy loading. The trends show linear scalability for both the loading time and throughput. In [14], we discussed the data access speed of our system compared with the legacy UDA API, showing a tenfold improvement in that metric.

VII. Discussion

Our solution provides a robust foundation for future data science projects working with data from UKAEA's facilities. In line with the FAIR principles [25], we have considerably enhanced the findability and accessibility of data from UKAEA facilities. Our solution involves a metadata database accompanied by web APIs, which allow users to locate relevant data efficiently. In addition, our public-facing object storage enables users to access MAST data without any authentication barriers. In terms of reusability and interoperability, we have provided a comprehensive set of data and metadata attributes, present in both the data files and the metadata database, as shown in Figs. 5 and 6. Furthermore, we have established a data curation pipeline to transform the data into formats with descriptive coordinates and dimensions, linking to the IMAS data dictionary names.

We have extended our prior work [14] further to enable ingestion of data from other facilities, including data from MAST-U and JET. We emphasize that this project's design, architecture, and software are generic and could be applied to other tokamak systems. Downstream users only need to provide a specific implementation for reading information from a legacy archive system (e.g., UDA/SAL) and providing a mapping file defining transformation, grouping, and naming. One further challenge not addressed in this work is the need to handle embargoed data and restrictive data sharing policies. Future development will add an authentication layer to the current system to enable permissive sharing of sensitive data to other facilities. Another future enhancement not addressed here is the ingestion of live data from an operating tokamak such as MAST-U. We aim to address the streaming of live diagnostic data directly into our system in the future.

We plan to further enhance the system's scalability and ensure rapid data access by incorporating extra mirrors for the data at other sites. We are working on setting up another Ceph storage solution at CSD3 in Cambridge [51] and are actively exploring the option of hosting a mirror of the data with the Amazon Software Sustainability Initiative [37]. Furthermore, we will incorporate more metadata from the IMAS data standard into our data products, as well as provide tooling to map from our data products into full IMAS data structures.

We are inspired by the possibilities for collaboration that these public data will enable. Work is already underway with the EPFL to apply their DEFUSE [52] event tagging tool to the MAST data. Such downstream projects are beneficial not only for the UKAEA and wider fusion energy community but also can potentially help improve the archive and metadata. For example, we foresee that tags for disruptions, sawteeth, MHD events, etc. derived from these data can be ingested as searchable metadata attributes or data artifacts in their own right.

VIII. Conclusion

We developed an open data service designed to make fusion data FAIR, machine learning ready, and open for the broader research community. By implementing a FAIRification pipeline, we have standardized the diagnostic history of the MAST, MAST-U, and JET experiments, making it readily accessible through web interfaces and APIs for both the interactive queries and bulk data access. Our system offers unrestricted public access to the MAST data supporting a wide range of applications, including machine learning, statistical analysis, and visual analytics. Furthermore, our system enables uniform access and sharing of MAST-U and JET data (given UKAEA network access).

The significance of our work extends beyond UKAEA's facilities. As the fusion community progresses from research-oriented devices to large-scale energy production facilities, leveraging historical experimental data is crucial for informing and improving future power plant designs. Our system provides a scalable and adaptable template for making fusion data open and FAIR. We believe that this work will not only facilitate cross-machine data analysis but also foster collaboration, accelerating the discovery of innovative solutions to the complex technical challenges in fusion research.

IX. Author Contributions

Samuel Jackson: designed and implemented the ingestion pipeline, database, and API, deployed the software, and wrote this article and figures; Saiful Khan: provided technical leadership, supervised the architecture and design, and wrote this article and figures; Nathan Cummings: provided the MAST data, fusion domain knowledge, and the project's requirements; James Hodson: assisted in developing the metadata database and API; Shaun de Witt: led a work package; Stanislas Pamela: served as the program manager and budget holder; and Rob Akers and Jeyan Thiyagalingam: initiated the collaboration.

Acknowledgment

The authors thank Stephen Dixon, Jonathan Hollocombe, Adam Parker, Lucy Kogan, and Jimmy Measures from UKAEA for their help with the data. They thank Deniza Chekrygina from STFC for providing object storage from the IRIS resource program. They also acknowledge the IAEA's five-year Coordinated Research Project, which aims to establish a FAIR and open fusion database, as the motivation

for undertaking this work. Samuel Jackson contributed the majority of this work at STFC before moving to UKAEA.

The MAST Team list are: A. Kirk, J. Adamek, R.J. Akers, S. Allan, L. Appel, F. Arese Lucini, M. Barnes, T. Barrett, N. Ben Ayed, W. Boeglin, J. Bradley, P.K. Browning, J. Brunner, P. Cahyna, S. Cardnell, M. Carr, F. Casson, M. Cecconello, C. Challis, I.T. Chapman, S. Chapman, J. Chorley, S. Conroy, N. Conway, W.A. Cooper, M. Cox, N. Crocker, B. Crowley, G. Cunningham, A. Danilov, D. Darrow, R. Dendy, D. Dickinson, W. Dorland, B. Dudson, D. Dunai, L. Easy, S. Elmore, M. Evans, T. Farley, N. Fedorczak, A. Field, G. Fishpool, I. Fitzgerald, M. Fox, S. Freethy, L. Garzotti, Y.C. Ghim, K. Gi, K. Gibson, M. Gorelenkova, W. Gracias, C. Gurl, W. Guttenfelder, C. Ham, J. Harrison, D. Harting, E. Havlickova, N. Hawkes, T. Hender, S. Henderson, E. Highcock, J. Hillesheim, B. Hnat, J. Horacek, J. Howard, D. Howell, B. Huang, K. Imada, M. Inomoto, R. Imazawa, O. Jones, K. Kadowaki, S. Kaye, D. Keeling, I. Klimek, M. Kocan, L. Kogan, M. Komm, W. Lai, J. Leddy, H. Leggate, J. Hollocombe, B. Lipschultz, S. Lisgo, Y.Q. Liu, B. Lloyd, B. Lomanowski, V. Lukin, I. Lupelli, G. Maddison, J. Madsen, J. Mailloux, R. Martin, G. McArdle, K. McClements, B. McMillan, A. Meakins, H. Meyer, C. Michael, F. Militello, J. Milnes, A.W. Morris, G. Motojima, D. Muir, G. Naylor, A. Nielsen, M. O'Brien, T. O'Gorman, M. O'Mullane, J. Olsen, J. Omotani, Y. Ono, S. Pamela, L. Pangione, F. Parra, A. Patel, W. Peebles, R. Perez, S. Pinches, L. Piron, M. Price, M. Reinke, P. Ricci, F. Riva, C. Roach, M. Romanelli, D. Ryan, S. Saarelma, A. Saveliev, R. Scannell, A. Schekochihin, S. Sharapov, R. Sharples, V. Shevchenko, K. Shinohara, S. Silburn, J. Simpson, A. Stanier, J. Storrs, H. Summers, Y. Takase, P. Tamain, H. Tanabe, H. Tanaka, K. Tani, D. Taylor, D. Thomas, N. Thomas-Davies, A. Thornton, M. Turnyanskiy, M. Valovic, R. Vann, F. Van Wyk, N. Walkden, T. Watanabe, H. Wilson, M. Wischmeier, T. Yamada, J. Young, S. Zoletnik, and the EUROfusion MST1 Team.

REFERENCES

[1] N. Holtkamp, "An overview of the ITER project," *Fusion Eng. Design*, vol. 82, nos. 5–14, pp. 427–434, Oct. 2007. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0920379607001160

[2] S. Ciattaglia, G. Federici, L. Barucca, A. Lampasi, S. Minucci, and I. Moscato, "The European DEMO fusion reactor: Design status and challenges from balance of plant point of view," in *Proc. IEEE Int. Conf. Environ. Electr. Eng. IEEE Ind. Commercial Power Syst. Eur.*, Jun. 2017, pp. 1–6. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/7977853

[3] T. Hey et al., *The Fourth Paradigm: Data-Intensive Scientific Discovery*. Redmond, WA, USA: Microsoft research, 2009.

[4] R. Anirudh et al., "2022 review of data-driven plasma science," *IEEE Trans. Plasma Sci.*, vol. 51, no. 7, pp. 1750–1838, Jul. 2023.

[5] A. Pavone, A. Merlo, S. Kwak, and J. Svensson, "Machine learning and Bayesian inference in nuclear fusion research: An overview," *Plasma Phys. Controlled Fusion*, vol. 65, no. 5, Apr. 2023, Art. no. 053001, doi: 10.1088/1361-6587/acc60f.

[6] C. Li et al., "Multimodal foundation models: From specialists to general-purpose assistants," 2023, *arXiv:2309.10020*.

[7] R. Bommasani et al., "On the opportunities and risks of foundation models," 2021, *arXiv:2108.07258*.

[8] A. Sykes et al., "First results from MAST," *Nucl. Fusion*, vol. 41, no. 10, pp. 1423–1433, Oct. 2001, doi: 10.1088/0029-5515/41/10/310.

[9] G. F. Counsell et al., "Overview of MAST results," *Nucl. Fusion*, vol. 45, no. 10, pp. S157–S167, Oct. 2005, doi: 10.1088/0029-5515/45/10/s13.

[10] H. Meyer et al., "Overview of physics results from MAST," *Nucl. Fusion*, vol. 49, no. 10, Sep. 2009, Art. no. 104017, doi: 10.1088/0029-5515/49/10/104017.

[11] J. R. Harrison et al., "Overview of new MAST physics in anticipation of first results from MAST upgrade," *Nucl. Fusion*, vol. 59, no. 11, Jun. 2019, Art. no. 112011, doi: 10.1088/1741-4326/ab121c.

[12] J. Bednar and M. Durant, "The pandata scalable open-source analysis stack," in *Proc. Python Sci. Conf.*, 2023, pp. 85–92. [Online]. Available: https://conference.scipy.org/proceedings/scipy2023/james_bednar.html

[13] M. D. Wilkinson et al., "The FAIR guiding principles for scientific data management and stewardship," *Scientific Data*, vol. 3, no. 1, Mar. 2016, Art. no. 160018, doi: 10.1038/sdata.2016.18.

[14] S. Jackson et al., "FAIR-MAST: A fusion device data management system," *SoftwareX*, vol. 27, Sep. 2024, Art. no. 101869.

[15] B. S. Sammuli et al., "TokSearch: A search engine for fusion experimental data," *Fusion Eng. Design*, vol. 129, pp. 12–15, Apr. 2018. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0920379618301042

[16] *TokSearch*. Accessed: May 28, 2025. [Online]. Available: https://ga-fdp.github.io/toksearch/latest/

[17] C. Wan, Z. Yu, X. Liu, X. Wen, X. Deng, and J. Li, "A robust and fast data management system for machine-learning research of tokamaks," *IEEE Trans. Plasma Sci.*, vol. 50, no. 12, pp. 4980–4986, Dec. 2022. [Online]. Available: https://ieeexplore.ieee.org/document/9972905/

[18] (2024). *25 Years of Massive Fusion Energy Experiment Data Open on the 'cloud' and Available To Everyone*. [Online]. Available: https://phys.org/news/2024-06-years-massive-fusion-energy-cloud.html

[19] S. Franke, L. Paulet, J. Schäfer, D. O'Connell, and M. M. Becker, "Plasma-MDS, a metadata schema for plasma science with examples from plasma technology," *Scientific Data*, vol. 7, no. 1, p. 439, Dec. 2020.

[20] (2020). *Dublin Core*. [Online]. Available: https://www.dublincore.org/schemas/

[21] (2023). *DCAT*. [Online]. Available: https://www.w3.org/TR/vocab-dcat-2/

[22] (2019). *DataCite*. [Online]. Available: https://schema.datacite.org/meta/kernel-4.2/

[23] *IMAS Data Dictionary Documentation*. Accessed: May 28, 2025. [Online]. Available: https://imas-data-dictionary.readthedocs.io/en/latest/

[24] S. Pinches. *Introduction To the Integrated Modelling & Analysis Suite (IMAS)*. Accessed: May 28, 2025. [Online]. Available: https://conferences.iaea.org/event/251/contributions/20713/attachments/11191/16492/IMAS%20Tutorial%20-%20Pinches.pdf

[25] *FAIR Principles*. Accessed: May 28, 2025. [Online]. Available: https://www.go-fair.org/fair-principles

[26] P. Strand et al., "A FAIR based approach to data sharing in Europe," *Plasma Phys. Controlled Fusion*, vol. 64, no. 10, Aug. 2022, Art. no. 104001, doi: 10.1088/1361-6587/ac8618.

[27] (Oct. 2024). *Ukaea/UDA*. [Online]. Available: https://github.com/ukaea/UDA

[28] (Jul. 2024). *Ukaea/fair-mast*. [Online]. Available: https://github.com/ukaea/fair-mast

[29] *Rasdaman*. Accessed: May 28, 2025. [Online]. Available: http://www.rasdaman.org/

[30] M. Stonebraker, P. Brown, A. Poliakov, and S. Raman, "The architecture of SciDB," in *Scientific and Statistical Database Management*. Berlin, Germany: Springer, 2011, pp. 1–16.

[31] H. Liu, P. van Oosterom, C. Hu, and W. Wang, "Managing large multidimensional array hydrologic datasets: A case study comparing NetCDF and SciDB," *Proc. Eng.*, vol. 154, pp. 207–214, Jan. 2016. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1877705816318380

[32] S. Papadopoulos, K. Datta, S. Madden, and T. Mattson, "The TileDB array data storage manager," *Proc. VLDB Endowment*, vol. 10, no. 4, pp. 349–360, Nov. 2016, doi: 10.14778/3025111.3025117.

[33] P. Baumann et al., "Big data analytics for Earth sciences: The Earth-Server approach," *Int. J. Digit. Earth*, vol. 9, no. 1, pp. 3–29, Jan. 2016, doi: 10.1080/17538947.2014.1003106.

[34] *HDF Format*. Accessed: May 28, 2025. [Online]. Available: https://www.hdfgroup.org/solutions/hdf5/

[35] *NetCDF Format*. Accessed: May 28, 2025. [Online]. Available: https://www.unidata.ucar.edu/software/netcdf

[36] *Zarr File Format*. Accessed: May 28, 2025. [Online]. Available: https://zarr.dev/

[37] *Amazon Sustainability Data Initiative*. Accessed: May 28, 2025. [Online]. Available: https://registry.opendata.aws/collab/asdi/

[38] *Tiledb Slicing Benchmarks*. Accessed: May 28, 2025. [Online]. Available: https://github.com/TileDB-Inc/tiledb-benchmarks/blob/master/bcolz-zarr/slicing-benchmarks.ipynb
[39] S. Hoyer and J. Hamman, "Xarray: N-D labeled arrays and datasets in Python," *J. Open Res. Softw.*, vol. 5, no. 1, p. 10, Apr. 2017, doi: 10.5334/jors.148.
[40] Kerchunk.*Kerchunk - Kerchunk Documentation*. Accessed: May 28, 2025. [Online]. Available: https://fsspec.github.io/kerchunk/
[41] *PostgreSQL JSON Types*. Accessed: May 28, 2025. [Online]. Available: https://www.postgresql.org/docs/current/datatype-json.html
[42] *GraphQL*. Accessed: May 28, 2025. [Online]. Available: https://spec.graphql.org/October2021/
[43] S. Khan, P. H. Nguyen, A. Abdul-Rahman, E. Freeman, C. Turkay, and M. Chen, "Rapid development of a data visualization service in an emergency response," *IEEE Trans. Services Comput.*, vol. 15, no. 3, pp. 1251–1264, May 2022.
[44] S. Khan, E. Rydow, S. Etemaditajbakhsh, K. Adamek, and W. Armour, "Web performance evaluation of high volume streaming data visualization," *IEEE Access*, vol. 11, pp. 15623–15636, 2023.
[45] S. Khan and D. Wallom, "A system for organizing, collecting, and presenting open-source intelligence," *J. Data, Inf. Manage.*, vol. 4, no. 2, pp. 107–117, Jun. 2022.
[46] *Pint: Makes Units Easy - Pint Documentation*. Accessed: May 28, 2025. [Online]. Available: https://pint.readthedocs.io/en/stable/
[47] *MAST Catalog*. Accessed: May 28, 2025. [Online]. Available: https://mastapp.site/intake/catalog.yml
[48] *Ceph*. Accessed: May 28, 2025. [Online]. Available: https://dl.acm.org/doi/abs/10.5555/1369043
[49] (Apr. 2025). *Dask*. [Online]. Available: https://github.com/dask/dask
[50] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," 2019, *arXiv:1912.01703*.
[51] *Cambridge Service for Data Driven Discovery (CSD3)*. Accessed: May 28, 2025. [Online]. Available: https://www.csd3.cam.ac.uk/
[52] X. Litaudon et al., "EUROfusion contributions to ITER nuclear operation," *Nucl. Fusion*, vol. 64, no. 11, Mar. 2024, Art. no. 112006. [Online]. Available: http://iopscience.iop.org/article/10.1088/1741-4326/ad346e

Samuel Jackson received the M.Eng. degree in software engineering from the University of Aberystwyth, Aberystwyth, U.K., in 2016.

He is a Principal Data Scientist with U.K. Atomic Energy Authority (UKAEA), Abingdon, U.K. He has previously worked in various roles within the U.K.'s large-scale experimental facilities and national laboratories. He has been involved in numerous projects toward improving scientific data analysis and reduction workflows. His expertise is in machine learning, software engineering, and high-performance computing.

Saiful Khan received the D.Phil. degree in engineering science from the University of Oxford, Oxford, U.K., in 2015.

He was a Post-Doctoral Research with the University of Oxford. He is a Senior Data Scientist with STFC, Swindon, U.K. He has worked as a Data Scientist with Horus Security Consultancy and International Seismological Centre, Oxford, and as a Software Engineer with Oracle, Bengaluru, India, and ABB, Bengaluru. His research interests include data-driven system development involving machine learning and data visualization.

Nathan Cummings received the M.Phys. degree in physics from the University of Bath, Bath, U.K., in 2019.

He is a Senior Data Engineer with U.K. Atomic Energy Authority (UKAEA), Abingdon, U.K. He works with various members of the international fusion community to drive the development and adoption of standards as well as using community-developed tools to improve the utility of data for data-intensive applications such as AI/ML pipelines. His research interests include around data management and advancing the application of FAIR data principles to fusion data.

James Hodson received the B.Sc. degree in physics from the University of Bath, Bath, U.K., in 2021.

He is a Data Engineer with U.K. Atomic Energy Authority (UKAEA), Abingdon, U.K., with a background in physics and a strong focus on data-driven solutions for fusion energy research, where he began his career as a Radiation Analyst working on safety analysis for fusion-related projects. Over time, he transitioned into his current role, where he his involved with advancing data management practices within the organization.

Shaun de Witt has worked with U.K. Atomic Energy Authority (UKAEA), Abingdon, U.K., for 12 years and with organizations like NASA, CERN, and European Space Agency. He has developed tools for data access and management and helped define policies based on user requirements and legal frameworks. Recently, he has focused on the EUDAT project, supporting communities in establishing data management policies and promoting the FAIR principles for open data in collaboration with the FAIR Datapoint Project and the Digital Curation Centre.

Stanislas Pamela is a Researcher with U.K. Atomic Energy Authority (UKAEA), Abingdon, U.K., the National Laboratory for Fusion Research. He is the Head of the AI and Machine Learning Group and has worked in many areas of fusion in the past. In the past roles, he has also worked on advanced visualization techniques, experimental data analysis systems, finite-element methods, and magnetohydrodynamics. His research interests include machine learning applications for fusion, including both simulation data and experimental data, particularly using AI methods to accelerate conventional simulation solvers.

Rob Akers is the Director of Computing Programs and a Senior Fellow with U.K. Atomic Energy Authority (UKAEA), Abingdon, U.K. UKAEA is now focused on delivering the world's first commercial-scale fusion power plant, STEP. He leads initiatives to use exascale supercomputing, AI, and big data across various advanced programs in simulation and data science. His team of more than 200 experts collaborates with around 100 organizations, including hyperscalers, HPC manufacturers, academic institutes, and specialized SMEs.

Jeyan Thiyagalingam (Senior Member, IEEE) received the Ph.D. degree in computer science from Imperial College, London, U.K., in 2005.

He is the Technical Lead for the AI for Science, RAL, STFC, Swindon, U.K. He was a Faculty Member with the University of Liverpool, Liverpool, U.K., within the School of Electrical and Electronic Engineering and Computer Science, and worked in industries, including MathWorks, Natick, MA, USA. His research interests and expertise are in AI for science, data processing algorithms, and signal processing.

Dr. Thiyagalingam is a fellow of British Computer Society.